

데이터통신 프로그래밍 · HW#1

분산 환경에서의 Fault-Tolerant 키-값 저장소 구현

Multi-Thread 소켓 통신 · 동적 부하 분산 · 장애 복구(P2P)



제출 마감 (DUE)

2026. 9. 23(수) 23:59



제출 방법

G조이름HW1.zip 1개 파일 (조별 1명만 제출, 예: G1HW1.zip)

수업용 해설 자료 · Assigned 2026.9.10(목) 09:00

OVERVIEW

오늘 살펴볼 내용



01

핵심 개념

분산 키-값 저장소, Master/Worker 역할



02

시스템 구조

Master-Worker 아키텍처 & 소켓 연결



03

동적 작업 분배

Queue 상태 기반 작업 할당 알고리즘



04

P2P 부하 분산

Worker 간 직접 작업 이전



05

장애 복구

Fault Tolerance 처리 흐름



06

시뮬레이션 흐름

Step 1~5 전체 시나리오



07

평가지표 & 로그

성능 지표와 로그 출력 형식



08

제출물 체크리스트

zip 구성과 필수 구현 조건

과제 한눈에 보기

무엇을 만드나요?

1개의 Master Node와 4개의 Worker Node로 구성된 분산 키-값 저장소를 직접 구현합니다. 모든 노드는 Socket(TCP) 통신과 독립 Thread로 동작하며, 5,000개의 (Key, Value) 작업을 Master가 Worker들에게 동적으로 분배하고, Worker들은 서로 P2P로 부하를 나누며, 장애가 발생하면 자동으로 복구합니다.



Socket(TCP) 통신



Thread 기반 동작



동적 분배 + P2P



Fault Tolerance

제출 개요



DUE

2026.9.23(수) 23:59



Assigned

2026.9.10(목) 09:00



제출 파일

G조이름HW1.zip (1개, 조별 1명)



주의

압축 파일 오류 시 0점 처리

CORE IDEA

분산 키-값 저장소란?

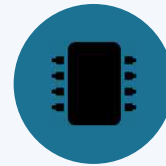
분산 키-값 저장소(Distributed Key-Value Store)는 대규모 데이터를 여러 노드에 나누어 저장하고 고가용성을 보장하는 저장 시스템입니다. 이번 과제는 이를 Master-Worker 구조로 단순화한 버전입니다.



Master Node

“지휘자 — 데이터를 나누고, 문제를 해결한다”

- System Clock 관리 및 5,000개 KV 쌍 생성
- Worker Queue 상태를 모니터링하여 작업 동적 분배
- Worker 실패 보고 시 Priority Queue로 재할당
- 반드시 AWS/GCP 등 외부 서버에 구현



Worker Node (×4)

“실무자 — 작업을 처리하고, 서로 돕는다”

- Ready Queue(최대 10개)에서 작업 수신 및 처리
- 작업당 1~3초 처리, 성공률 80% 시뮬레이션
- 큐 대기 임계값(15초) 초과 시 P2P로 이웃에 이전
- 독립적인 Thread로 각각 동작 (조원 로컬 PC에서 실행, Master IP·포트로 접속)

KEY TERMS

핵심 개념 정리



Key-Value Store

Key는 고유 16진수 4자리, Value는 1~100 정수.
Map 기반 O(1) 접근.



Master Node

System Clock 관리, 5,000개 KV 생성, 작업 동적 분
배, 장애 감지·재할당.



Worker Node

Ready Queue(최대10)로 작업 수신, 1~3초 처리,
80% 성공률, P2P 참여.



Ready Queue

워커의 대기 작업 버퍼. 최대 10개, 초과 요청은 즉
시 FAIL 처리.



System Clock

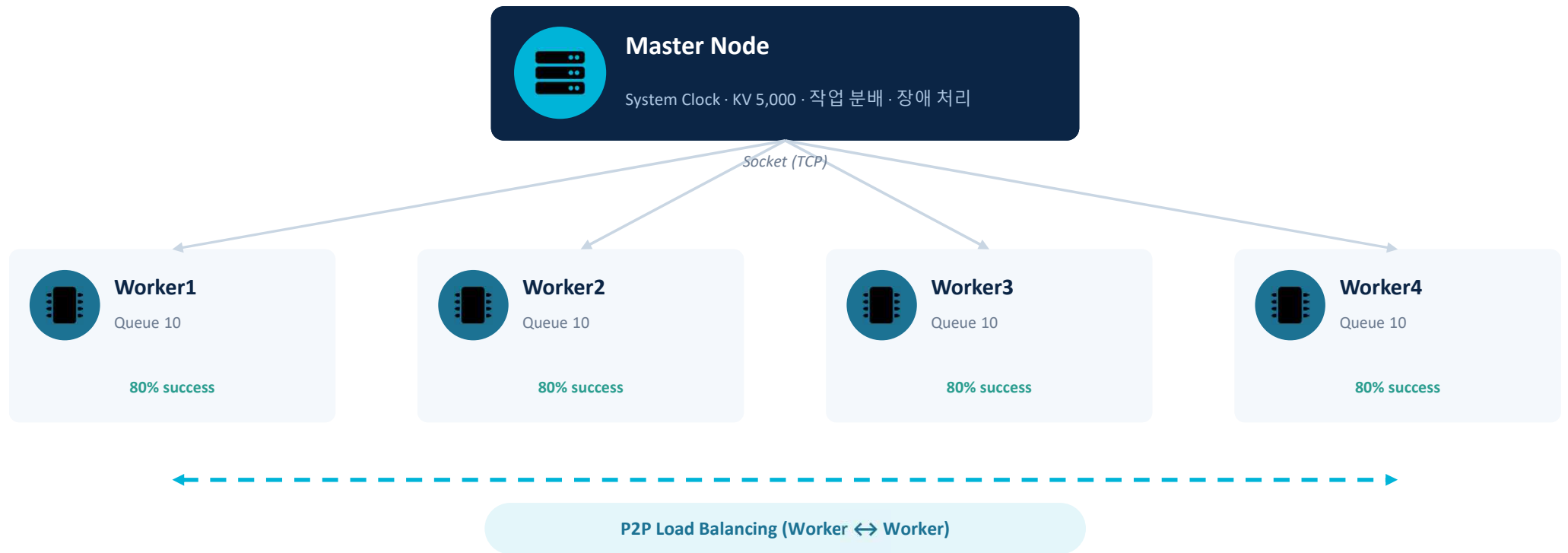
실제 시간이 아닌 가상의 누적 시간(sec). 모든 이
벤트 로그에 함께 기록.



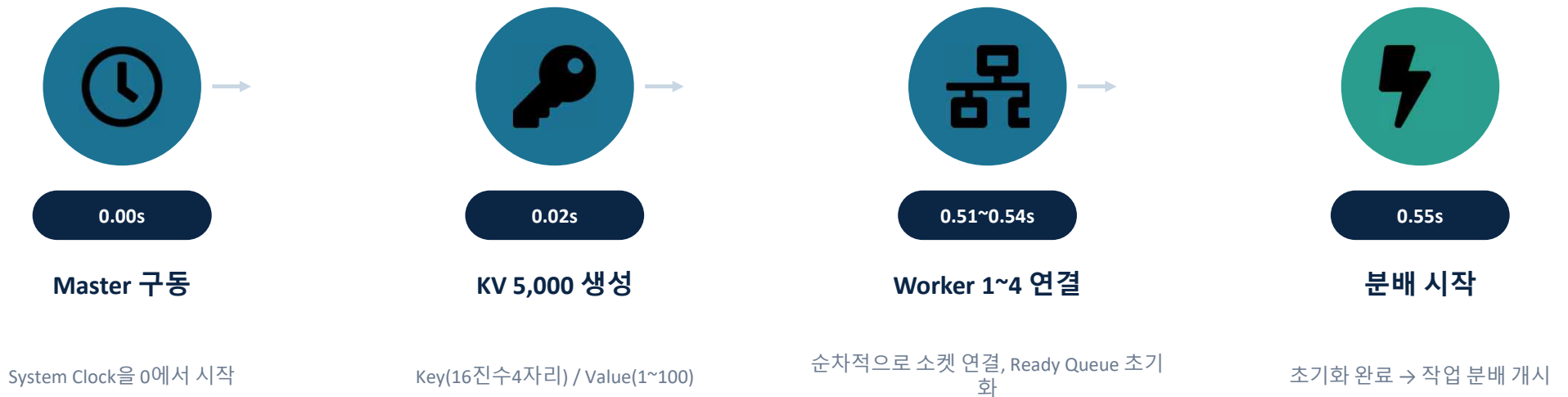
Fault Tolerance

Worker 실패(20%) 시 Master가 감지, 다른 Worker
에 재할당하여 유실 방지.

시스템 구조 & 데이터 흐름



초기화 (System Clock : 0 sec)



핵심: Master가 먼저 시작해 데이터를 준비하고, 4개의 Worker가 순서대로 접속을 완료해야 비로소 작업 분배(Step 2)가 시작됩니다.

동적 작업 분배 알고리즘 예시

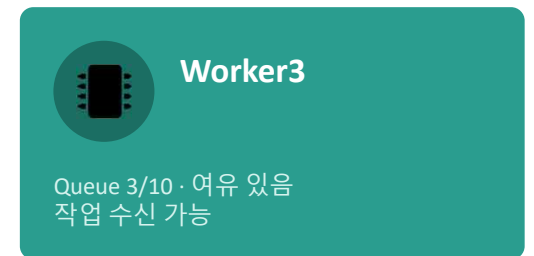
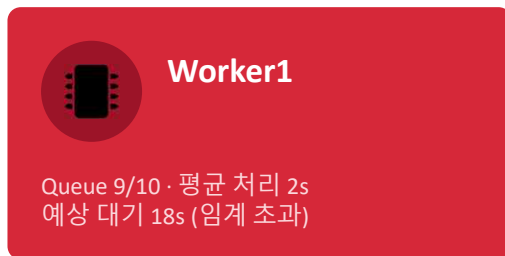
Master는 각 Worker의 Queue 상태를 지속적으로 모니터링하여, 여유 공간이 가장 많은 Worker에게 우선적으로 작업을 할당합니다.

시점	Worker1 Queue	Worker2 Queue	Worker3 Queue	Master 판단
t = 1s	3 / 10	2 / 10	1 / 10	Worker3에 할당 (최소 큐)
t = 5s	8 / 10	5 / 10	2 / 10	Worker3 우선 할당 / Worker1 포화 주의
t = 9s	10 / 10	9 / 10	4 / 10	Worker1 큐 초과 → 해당 요청 FAIL 처리
t = 12s	FAIL	6 / 10	3 / 10	Worker1 장애 → 작업 Worker3에 재할당

규칙: Worker Queue가 최대(10개)에 도달하면 해당 Worker로의 분배를 일시 중단하고, 통신 지연 1sec를 가정하여 전체 5,000개 작업을 반복 분배합니다.

P2P 부하 분산 동작 예시

Worker 노드끼리 직접 소켓 통신으로 부하를 분산합니다. 예상 대기시간(큐에 남은 작업 수 × 평균 처리시간 2초)이 임계값(15sec)을 초과하면 인접 Worker에게 작업 이전을 요청하며, 이 값은 1~3초 랜덤 주기로 점검합니다.



- ① 대기시간 모니터링 → 임계 초과 감지
- ② 인접 Worker2:3 큐 상태 조회
- ③ Worker3로 작업 3건 이전 요청 (P2P 소켓)
- ④ Worker3 수신 후 큐 추가, ACK 응답
- ⑤ Worker1 ACK 수신 → 큐 제거, 균형 복원

장애 처리 (Fault Tolerance) 흐름

예시: Worker2가 작업 처리 중 실패(20% 확률)를 Master에 보고하는 경우



Worker2 처리 실패

KV 쌍 저장 중 실패 발생 (80/20 확률 시뮬레이션)



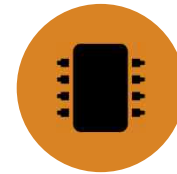
Master에 FAIL 보고

Worker2 → Master : FAIL 메시지 전송



Priority Queue 등록

Master가 실패한 작업 ID를 최우선 큐에 등록



여유 Worker에 재할당

가장 큐가 여유로운 Worker(예: Worker4) 선택



재처리 & 결과 갱신

Worker4가 SUCCESS 응답 → KV-진행률 갱신

포인트: 재할당된 작업도 80%/20% 성공·실패 규칙이 동일하게 적용되며(다시 실패하면 재재할당), 기존 분배와 동일한 방식이되 항상 최우선 (Priority) 순위로 처리됩니다.

BIG PICTURE

시뮬레이션 전체 흐름 (Step 1~5)



Step 1

초기화



Step 2

동적 작업 분배



Step 3

작업 처리 & 수집



Step 4

P2P 부하 분산



Step 5

종료 & 통계

Master 시작 → KV 5,000 생성 → Worker 4개 연결 Queue 여유가 가장 많은 Worker에 우선 할당 1~3초 처리, 80% 성공 / 20% 실패 시뮬레이션 대기 15초 초과 시 인접 Worker에게 작업 이전 5,000건 완료 → 통계 로그 후 Graceful 종료

이 5 단계가 195.40초(가상 시간, 예시) 동안 반복 병행되며 전체 로그(Master.txt, Worker1~4.txt)에 기록됩니다. (실제 프로그램 실행은 sleep 없이 훨씬 빠르게 진행됩니다)

CRITICAL REQUIREMENTS

필수 구현 조건



하나라도 미충족 시 과제 전체 0점 처리됩니다



외부 서버 필수

Master Node는 반드시 AWS, GCP 등 물리적 외부 클라우드 서버에 구현되어야 합니다.



Socket 통신 필수

Master $\leftrightarrow$ Worker, Worker $\leftrightarrow$ Worker 간 모든 통신은 반드시 Socket으로 구현해야 합니다.



독립 Thread 필수

모든 Worker Node는 각각 독립적인 Thread를 통해 동작해야 합니다.

추가로 동적 작업 분배 알고리즘과 P2P 부하 분산 알고리즘은 자유롭게 설계하되, 제출 시 사용한 알고리즘을 Readme에 반드시 명시해야 합니다.

EVALUATION METRICS

성능 평가 지표

다음 5가지 지표를 측정하여 각 노드의 최종 로그에 기록해야 합니다.



Worker별 작업 처리량

각 Worker가 성공적으로 처리한 KV 쌍의 수

건



Worker별 성공/실패 횟수

전체 처리 중 성공 및 20% 실패 케이스 카운트

건



작업 평균 대기시간

Queue에 들어온 후 처리 시작까지의 평균 대기 시간

sec



P2P 부하 분산 이벤트 횟수

Worker 간 작업 이전이 발생한 총 횟수

회



장애 재할당 횟수

실패로 인해 다른 Worker에 재할당된 작업 수

건



전체 수행시간

System Clock 기준 전체 시나리오 소요 시간

sec

LOGGING

노드별 Log 출력 형식

각 노드(Master, Worker1~4)는 모든 이벤트를 System Clock과 함께 아래 공통 형식으로 기록합니다.

[clock] NODE | EVENT | STATUS | message

SUCCESS

FAIL

WARN

INFO

EVENT(자유형식)

```
[0.00] MASTER | INIT      | SUCCESS | System Clock started.

[0.51] MASTER | CONNECT  | SUCCESS | Worker1 connected. Ready Queue initialized (0/10).

[4.10] MASTER | RESULT   | FAIL    | KV[0003] FAILED by Worker2 (20% rule). Requeueing...

[4.12] MASTER | DISTRIB  | SUCCESS | KV[0003] reassigned to Worker4.

[9.00] MASTER | DISTRIB  | WARN    | Worker1 queue full (10/10). Pausing dispatch.

[195.45] MASTER | TERMINATE | SUCCESS | Graceful shutdown. All workers disconnected.
```

Master.txt 로그 예시(발췌)

SUBMISSION

과제 제출물 체크리스트

G조이름HW1.zip (예: G1HW1.zip) — 압축 파일 오류 시 0점 처리



전체 소스 코드

프로그램 전체 소스코드



download.txt

5분 이내 시연 영상(G조이름HW1.mp4) 다운로드 링크



AllDefinedLogs.txt

정의한 모든 로그 메시지 명세 및 설명



Readme.txt

조원정보 · 구성요소 · 실행방법 · 분배/p2p 알고리즘 설명 · 장애처리 설명



결과 로그 5종

Master.txt, Worker1~4.txt — 각 노드 전체 로그

⚠ 소스코드 컴파일/실행 불가 시 과제 전체 0점 처리

WRAP-UP

오늘의 핵심 정리



외부 서버 + Socket + Thread는 선택이 아닌 필수

AWS/GCP 서버, Socket 통신, 독립 Thread — 하나라도 빠지면 0점입니다.



알고리즘은 직접 설계하고 반드시 명시

동적 작업 분배와 P2P 부하 분산 알고리즘을 설계하여 Readme에 기술하세요.



로그 5종 + 시연 영상까지가 결과물

Master/Worker 로그와 AllDefinedLogs.txt, 5분 이내 시연 영상을 함께 제출합니다.

제출 마감: 2026년 9월 23일(수) 23:59 — 미리 준비하세요!